Programming in Java – Assignment 1 (CO1)
Campus Smart-Parking System
TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

---

1. Problem Analysis

The university campus has a multi-level parking facility used by students, staff and visitors. Each floor has a fixed number of numbered slots, and the facility may be as small as 1 floor × 20 slots or as large as 5 floors × 100 slots. The vehicles are two-wheelers, cars and electric vehicles (EVs); EVs may also use charging slots, which are charged separately.

The system has two core operations. On **entry**, it must allocate a suitable free slot, issue a ticket with a unique number and record the entry time. On **exit**, it must release the slot and compute the fee from the vehicle type and the duration of parking. Students and staff with a valid monthly pass park free of charge, while visitors pay the full fee. Since the facility size and the pricing rules are expected to grow, the program must be modelled as a set of objects with clear responsibilities, so that new rules can be added without rewriting classes that already work.

2. Inputs and Outputs of the System

Table 1: Inputs and outputs for each operation

Operation	Inputs received	Outputs produced
Facility setup	Number of floors; number of slots on each floor; hourly rate of each vehicle type; list of valid monthly passes	A ready facility in which every slot is free
Vehicle entry	Registration number; vehicle type (two-wheeler, car, EV); user category (student, staff, visitor); entry time	Parking ticket with a unique ticket number, floor, slot number and entry time; or a “facility full” or “request rejected” message with the reason
Vehicle exit	Ticket number (or registration number); exit time; for an EV, the units charged in kWh	Parking duration; parking fee (zero for a valid pass); charging fee; total amount payable; slot released; ticket closed
Enquiry and report	Request for occupancy or for the day’s collection	Free slots on each floor; number of vehicles parked; total collection for the day

3. Candidate Classes (Nouns) and Their Justification

The nouns in the problem statement were examined one by one. A noun becomes a class only if it has both data and behaviour of its own; otherwise it is kept as a field of another class.

Table 2: Noun analysis

Noun in the problem	Decision	Justification
Vehicle (two-wheeler, car, EV)	Abstract class Vehicle; subclasses TwoWheeler, Car, ElectricCar	All vehicles share a registration number and the idea of a fee, but each has a different hourly rate. A common superclass avoids repeated code, and subclasses allow a new vehicle type to be added without changing existing classes.
Parking ticket	Class ParkingTicket	The ticket has its own data (ticket number, slot, entry and exit time, fee) and must outlive the moment of allocation, because it is needed again at exit. It is also the object that carries the unique ticket number.
Parking facility	Class ParkingFacility	Someone must own all the slots. Placing the slot structure in one class lets the rule “a slot cannot hold two vehicles” be enforced in one place only.
Floor and slot	Not separate classes; represented as slots[floor][slot] inside ParkingFacility	A slot is either free or holds one vehicle and has no behaviour of its own, so a class would add objects without adding responsibilities. This decision is revisited if slot types (charging bay, reserved) are introduced.
User (student, staff, visitor)	Abstract class ParkingUser; subclasses Student, Staff, Visitor	The fee rule depends on who the person is, not on the vehicle. Separate classes let the free-parking rule differ by category (visitors can never be exempt).
Monthly pass	Class MonthlyPass	A pass has its own data (pass number, valid-from and valid-to dates) and behaviour (is it valid on a given date?). An expired pass must not give free parking, so validity needs a home.
Fee / charge	Class FeeCalculator	Pricing policy changes more often than vehicles do. Keeping the combination of base fee, pass exemption and charging fee in one class prevents pricing rules from being scattered across the program.
Charging capability	Interface Chargeable	Charging is a capability that only EVs have. An interface states this capability without forcing twowheelers and ordinary cars to carry it.

Entry time, exit time, duration, fee amount, ticket number, registration number, slot number	Fields (attributes)	These are simple values that describe an object and have no behaviour of their own.
Noun in the problem	**Decision**	**Justification**
Campus, university, system	Not classes	They describe the boundary of the problem. A driver class, ParkingApp, reads the input, calls the model and prints the output.

4. Responsibilities of Each Class Table

3: Data held and responsibilities

Class	Data it holds	Responsibilities
Vehicle (abstract)	Registration number	Declares the abstract hourly rate; computes the base fee for a given duration (every started hour is charged); treats two vehicles with the same registration number as equal.
TwoWheeler, Car	Inherited registration number	Supply their own hourly rate by overriding the rate method.
ElectricCar	Inherited registration number	Behaves as a Car and, through Chargeable, also reports its charging fee for the units consumed.
Chargeable (interface)	None	Contract that any chargeable vehicle must provide a charging fee for a number of kWh.
ParkingTicket	Ticket number, registration number, floor, slot, entry time, exit time, fee	Records the parking session; knows whether it is still open; is closed exactly once at exit.
ParkingFacility	Two-dimensional slot structure; ticket counter; list of active tickets	Allocates the first free slot; releases a slot after validating its position; rejects entry when the facility is full or the vehicle is already parked; issues ticket numbers that are never reused; reports occupancy.

ParkingUser, Student, Staff, Visitor	Name or identifier; MonthlyPass (students and staff only)	States the user category and whether the user is eligible for free parking.
MonthlyPass	Pass number, valid-from date, valid-to date	Tells whether the pass is valid on a given date.
FeeCalculator	Rate and slab settings	Combines the vehicle's base fee, the user's pass exemption, the charging fee and any daily maximum into the final amount.
ParkingApp	None (uses the objects above)	Reads input, calls the model and displays output. It contains no business rules.

5. Verbs of the Problem and the Methods They Become

The action words in the problem statement identify the operations. Each one is assigned to the class that owns the data it needs. The method names below are proposed names that are used consistently in the later tasks.

Table 4: Verb to method mapping

Verb in the problem	Method	Owner class
Enter / park	allocate(String vehicleNo) – returns {floor, slot}	ParkingFacility
Issue a ticket	issueTicket(vehicle, floor, slot, entryTime)	ParkingFacility (creates a ParkingTicket)
Record entry time	Constructor and getEntryTime()	ParkingTicket
Leave / release	release(int floor, int slot) – returns true or false	ParkingFacility
Close the ticket	close(exitTime)	ParkingTicket
Compute the fee	calculateFee(int minutes); totalFee(...)	Vehicle; FeeCalculator
Hold a valid pass	isValid(date)	MonthlyPass
Charge an EV	chargingFee(int kWh)	Chargeable (implemented by ElectricCar)
Find a vehicle	findTicket(String regNo)	ParkingFacility
Report	availableSlots(); dailyCollection()	ParkingFacility

6. Business Rules Every Valid Operation Must Satisfy

Table 5: Business rules

No.	Rule	Enforced by
BR1	A slot cannot hold two vehicles; a slot is allocated only if it is currently free.	ParkingFacility.allocate()
BR2	A vehicle cannot be parked twice at the same time; the registration number must be unique among active tickets.	ParkingFacility (check before allocating)
BR3	A ticket number is unique and is never reused, even after the vehicle leaves. The counter only increases.	Ticket counter in ParkingFacility
BR4	If no suitable slot is free, entry is refused and no ticket is issued.	ParkingFacility.allocate() returns null
BR5	The floor and slot numbers must lie within the facility; only an occupied slot can be released, and only once.	ParkingFacility.release()
No.	**Rule**	**Enforced by**
BR6	The exit time cannot be earlier than the entry time, and a ticket can be closed only once.	ParkingTicket.close()
BR7	The fee depends on the vehicle type and the duration; every started hour is charged in full; a fee is never negative.	Vehicle.calculateFee()
BR8	Students and staff with a valid monthly pass pay no parking fee; visitors always pay the full fee; an expired pass gives no exemption.	FeeCalculator with MonthlyPass
BR9	EV charging is charged separately from parking, on the units consumed.	Chargeable.chargingFee()
BR10	The vehicle type must be one of the recognised types and the registration number must not be empty; otherwise the request is rejected.	ParkingApp and constructors

7. Assumptions About the Facility

- The facility is created once with a fixed number of floors and slots on each floor (for example 1 × 20, 3 × 50 or 5 × 100) and floors may have different slot counts. Floors and slots are numbered from 0 inside the program.
- Any free slot can take any vehicle type. Slots reserved for a particular type are not modelled in the base design.
- The allocation policy is “first free slot”, searching from the lowest floor and the lowest slot number.
- An EV can charge at its parking slot. Charging is billed on the units consumed (kWh) in addition to the parking fee, and a monthly pass does not waive the charging fee.
- Hourly rates follow the values used in the later “Complete the Code” tasks (two-wheeler Rs 10, car Rs 30, EV charging Rs 12 per kWh). Time is measured in whole

minutes and every started hour is billed in full. A daily maximum applies as a fee slab, to be defined in Task 3.

- Ticket numbers start at 1 and increase by 1 for every ticket issued.
- Whether a person is a student, staff member or visitor is given at entry, and pass validity is checked against the date of entry.
- The program runs in one thread with a single entry and exit point, and data is kept in memory only (no database or payment gateway).

8. Alternative Concept and Complexity Analysis

Alternative: store each vehicle as plain Strings in parallel arrays instead of as an object. Index i of every array describes the same vehicle.

```
String[] regNo     = new String[MAX];
String[] type      = new String[MAX];
String[] entryTime = new String[MAX]; int
count = 0;

int find(String reg) { for (int i = 0; i <
    count; i++) if (regNo[i].equals(reg))
    return i;
    return -1;
}
```

Here entryTime[] is a String[] so that all three arrays are plain Strings, exactly as in the stated alternative; the entry time is kept as text such as “09:15” and must be parsed into minutes at exit before the fee can be computed (a constant-time step for fixed-length text). A long[] of numeric timestamps would avoid the parsing and take 8 bytes per vehicle, but it would no longer be the all-String alternative that is being compared.

In the object design the same data is held as Vehicle[] parked, and the search is the same loop calling parked[i].getRegNo().equals(reg). The type is carried by the class (Car, TwoWheeler) instead of a type String, the entry time is held by the vehicle’s ParkingTicket (Table 3) as a numeric value, and fee calculation is done by the object itself. In the parallel-array design, the fee needs an if–else or switch on the type String, and a misspelt type such as “Cra” is accepted silently.

Table 6: Comparison of the two designs (n vehicles, registration number of constant length; memory figures are estimates, not measurements)

Criterion	Vehicle objects (chosen design)	Parallel String arrays
Find a vehicle by registration number	Linear search over Vehicle[]: worst case O(n), best case O(1). The object design can be indexed with a HashMap keyed by registration number to reach O(1) on average, with the vehicle data staying in one place.	Linear search over regNo[]: worst case O(n), best case O(1). A faster index needs a separate structure that must be kept in step with the three arrays.

Add a new vehicle attribute (for example user category)	Add one field, one constructor parameter and one getter in Vehicle. Methods that do not use the attribute remain unchanged. Time to add per vehicle is unchanged.	Declare a new array and update every place that inserts, removes, shifts or sorts vehicles, otherwise the arrays go out of step. Many methods must be edited and retested.
Criterion	**Vehicle objects (chosen design)**	**Parallel String arrays**
Space per vehicl (estimate, not measured)	Equivalent data only (registration number, type, entry time). The Vehicle object holds just the registration number: about 16 bytes (12-byte header + 4-byte reference). The type needs no field because the class itself carries it. The entry time is held in the vehicle's ParkingTicket (Table 3) as a numeric field of about 8 bytes. Total about 24 bytes per vehicle, which is O(1) per vehicle and O(n) overall. The ticket's own header and its other fields (ticket number, floor, slot, exit time, fee) have no counterpart in the three arrays, so they are left out of this comparison.	Equivalent data only. The three array slots hold 3 references: about 12 bytes. The type Strings are shared if they are literals, but each vehicle needs its own entryTime String such as "09:15", about 48 bytes (String object about 24 + its byte array about 24). Total about 60 bytes per vehicle, which is O(1) per vehicle and O(n) overall. Unused capacity up to MAX is also allocated in advance.
Safety and extensibility	Compiler checks types; a new vehicle type is a new subclass.	No type checking on the type String; every new type edits the fee code.

Basis of the memory figures. The byte counts are rough estimates for a typical 64-bit JVM with compressed references and compact Strings (Java 9 or later). They depend on the JVM version, its settings and object alignment, and they are not measured results. The registration number String is stored in both designs and costs the same in each, so it is not counted. Because the two designs store the entry time differently (numeric in the ticket, text in the array), the estimates compare the same information, not identical Java types.

Conclusion of the comparison. For finding a vehicle by registration number both designs are O(n) with a linear search, so the choice of objects does not cost anything asymptotically; the difference in space per vehicle is only a constant factor (estimated at about 24 bytes against about 60 bytes of equivalent data, depending on the JVM). The object design is clearly better when a new attribute or vehicle type is introduced, because the change is confined to one class, and this is the reason it is selected. No memory measurements have been made for this report; the byte figures are estimates and should be confirmed by measuring the actual program (for example with the Java Object Layout tool or Runtime memory methods) before they are quoted as results.

Conclusion

The problem was analysed in terms of inputs and outputs, and the candidate classes were identified from the nouns and their methods from the verbs, each with a stated reason. The classes are Vehicle with its subclasses, Chargeable, ParkingTicket, ParkingFacility, ParkingUser with its subclasses, MonthlyPass, FeeCalculator and ParkingApp. Ten business rules and eight assumptions were stated, and the parallel-array alternative was compared with the object design on search time, space and the cost of adding an attribute. These results are the basis for the data representation, control logic and class design in the following tasks.